Guía de Prompts para Desarrollo del Sistema

Índice de Contenidos

1. 1. Base de Datos
2. 2. Lógica de Negocio
3. 3. Servidor REST
4. 4. Lógica Fake (Aplicación Móvil)
5. 5. Lógica Fake (Cliente Web)
6. 6. Diseño de Interfaz Web

1. Base de Datos

Prompt:

Estoy haciendo un proyecto de Biometría y Medio Ambiente.

Ahora mismo quiero centrarme únicamente en la base de datos. Quiero que me generes la base de datos siguiendo este diseño. La tabla debe contener exactamente estos datos conceptuales

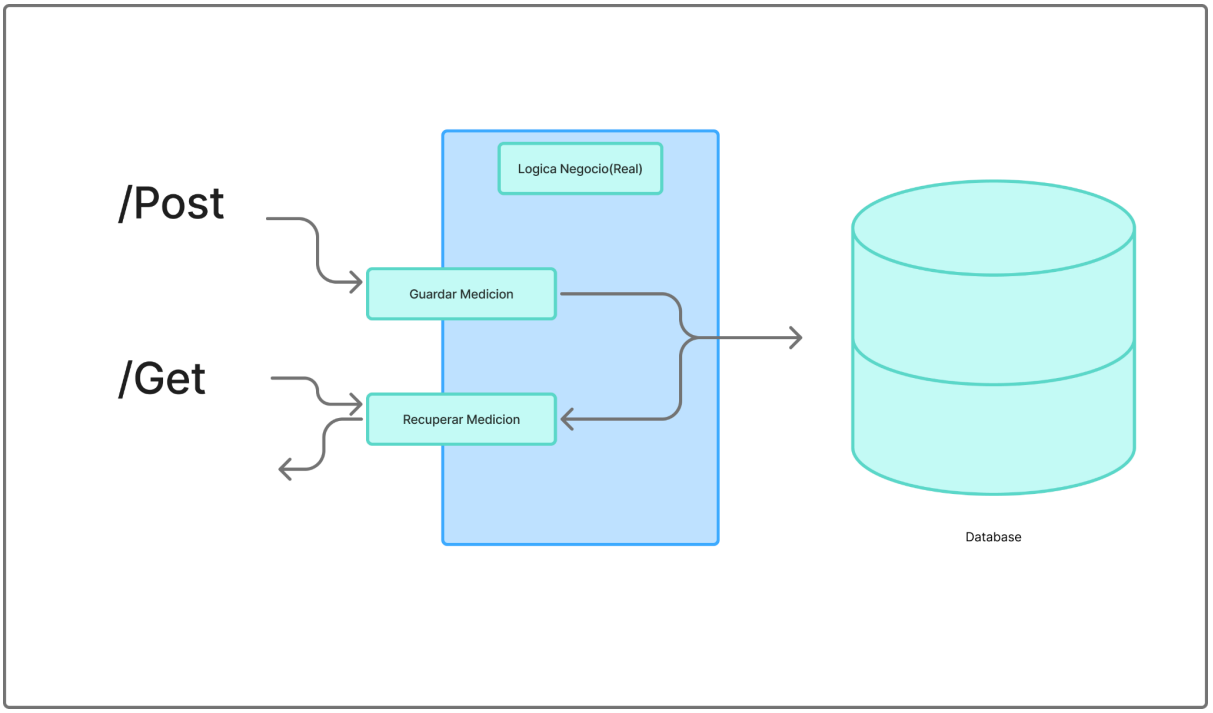
El diseño lógico de la tabla es:

```
Medicion(
   id : N,
   fecha : DateTime,
   tipo : Text,
   valor : R,
   major : N,
   minor : N
)
```

Quiero que se cumplan las siguientes restricciones:
- id debe ser la clave primaria.
- id debe generarse automáticamente al insertar una nueva medición.
- No deben permitirse valores nulos en esos campos.
- Incluye comentarios explicativos en el código SQL generado.

2. Lógica de Negocio

Prompt:



Estoy haciendo un proyecto de Biometría y Medio Ambiente.

En el backend quiero separar claramente:

- servidor REST;
- lógica de negocio;
- base de datos.

Ahora quiero que generes únicamente el código de la lógica de negocio real del backend.

Quiero seguir este diseño:
POST /medicion
↓
guardarMedicion()
↓
Base de datos
Base de datos
↓
recuperarMedicion()
↓
GET /medicion
Es decir:
- cuando llegue una petición POST, el servidor REST llamará a guardarMedicion();

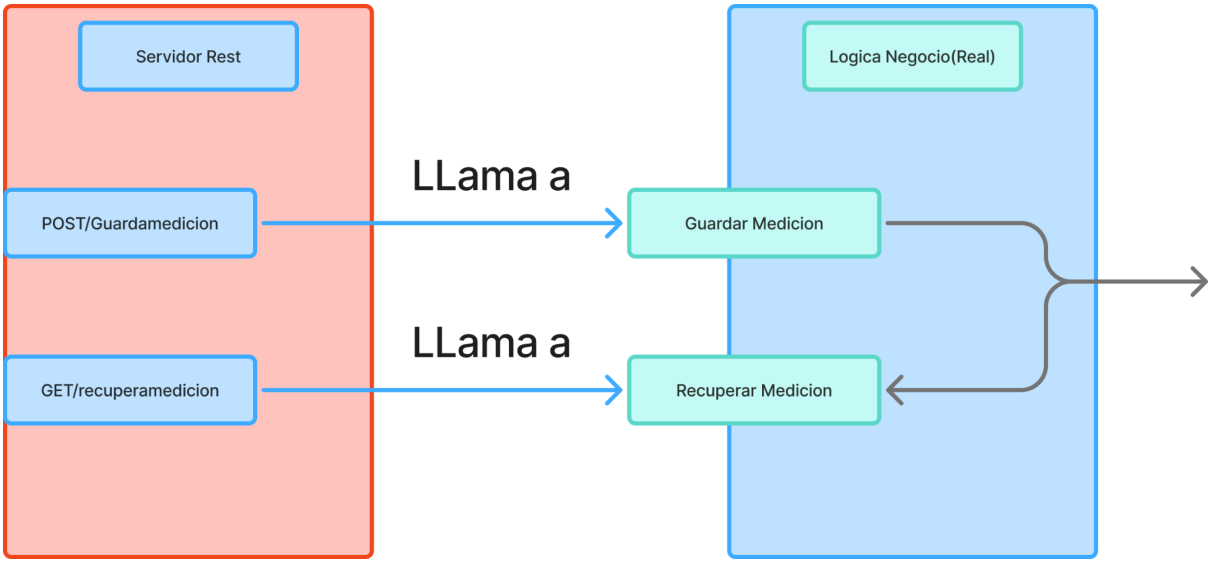
- guardarMedicion() se encargará de guardar la medición en la base de datos;
- cuando llegue una petición GET, el servidor REST llamará a recuperarMedicion();
- recuperarMedicion() consultará la base de datos y devolverá la última medición almacenada.

La lógica de negocio debe estar dentro de una clase llamada:
LogicaNegocio.
Y la tabla con la que va a interactuar en la base de datos es:
Medición(id, fecha, tipo, valor, major, minor).

- Añade comentarios explicativos claros en el código.
- Incluye una pequeña prueba automática (como un método main de prueba o test unitario) para verificar el correcto funcionamiento de guardarMedicion() y recuperarMedicion().

3. Servidor REST

Prompt:



Estoy haciendo un proyecto de Biometría y Medio Ambiente.

Quiero que generes únicamente el código del servidor REST del backend, teniendo en cuenta el siguiente dibujo.
El servidor REST debe estar separado de la lógica de negocio y únicamente debe encargarse de recibir las peticiones HTTP y llamar a los métodos correspondientes de la clase LogicaNegocio.

Según el diseño:

POST /Guardamedicion
↓

llama a
LogicaNegocio.guardarMedicion()

GET /recuperamedicion
↓
llama a
LogicaNegocio.recuperarMedicion()

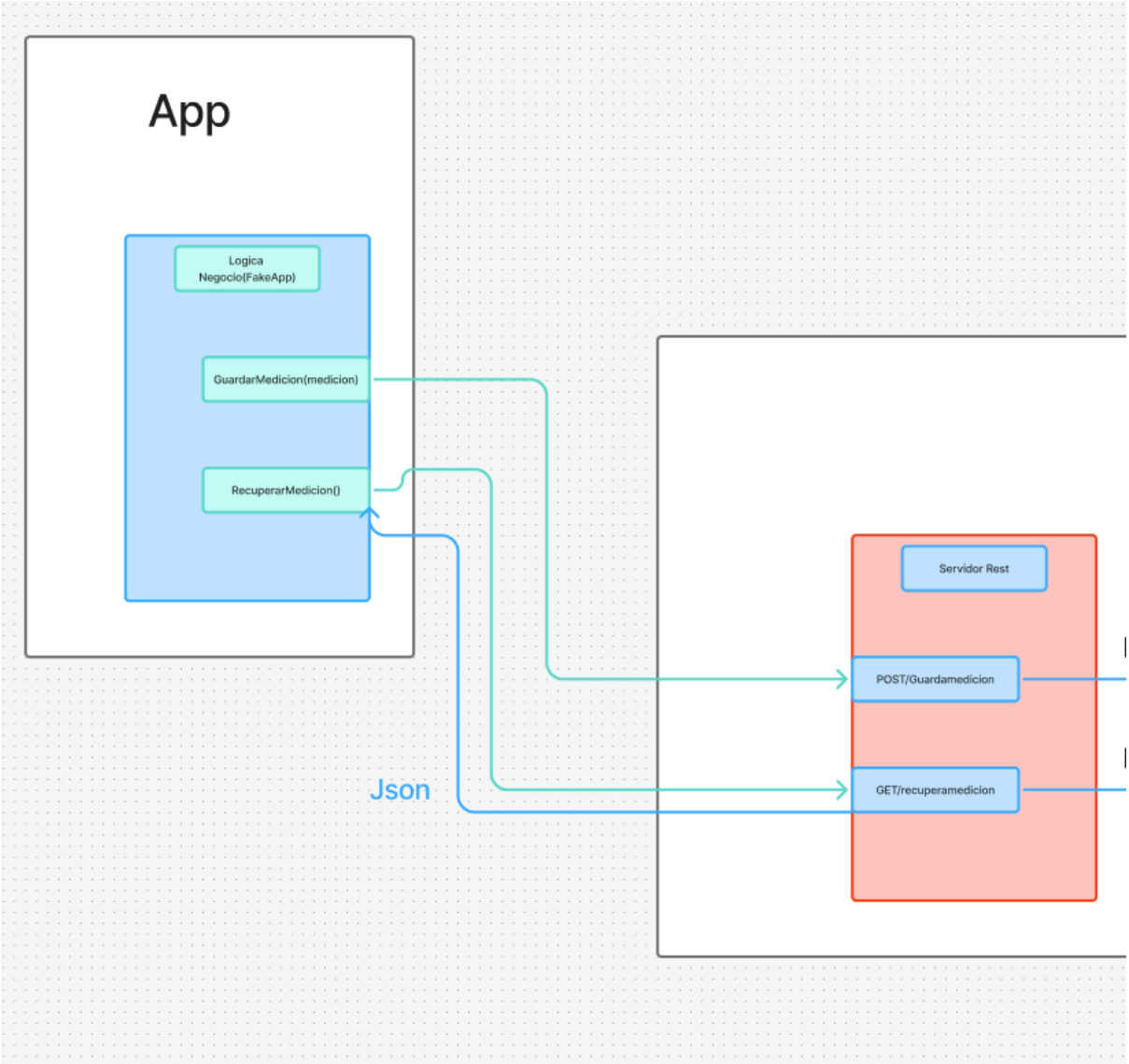
La ruta POST debe recibir una medición y enviársela a guardarMedicion().

La ruta GET debe llamar a recuperarMedicion() y devolver la última medición almacenada.

El servidor REST no debe acceder directamente a la base de datos ni contener la lógica real del sistema, ya que esa responsabilidad pertenece a LogicaNegocio.
- Incluye comentarios explicativos detallando el comportamiento de cada endpoint.
- Añade una pequeña prueba automática (por ejemplo, peticiones de prueba tipo cURL o un script sencillo) para verificar los endpoints POST y GET.

4. Lógica Fake (Aplicación Móvil)

Prompt:



Estoy haciendo un proyecto de Biometría y Medio Ambiente.

Ahora quiero que generes únicamente el código de la lógica fake que estará en la aplicación Android, teniendo en cuenta el siguiente dibujo.
La aplicación se desarrolla en Java usando Android Studio.

La lógica fake debe estar dentro de una clase llamada:
LogicaNegocioFakeApp

y tendrá únicamente estos dos métodos:
guardarMedicion(medicion)
recuperarMedicion()

El funcionamiento será:
guardarMedicion(medicion)
↓
POST /Guardamedicion

↓
Servidor REST

Servidor REST
↓
GET /recuperamedicion
↓
recuperarMedicion()
↓
devuelve la medición en JSON

La medición tiene esta estructura:

Medicion(id, fecha, tipo, valor, major, minor)

guardarMedicion() debe enviar la medición al servidor mediante una petición POST.

recuperarMedicion() debe realizar una petición GET y recibir la última medición enviada por el servidor en formato JSON.

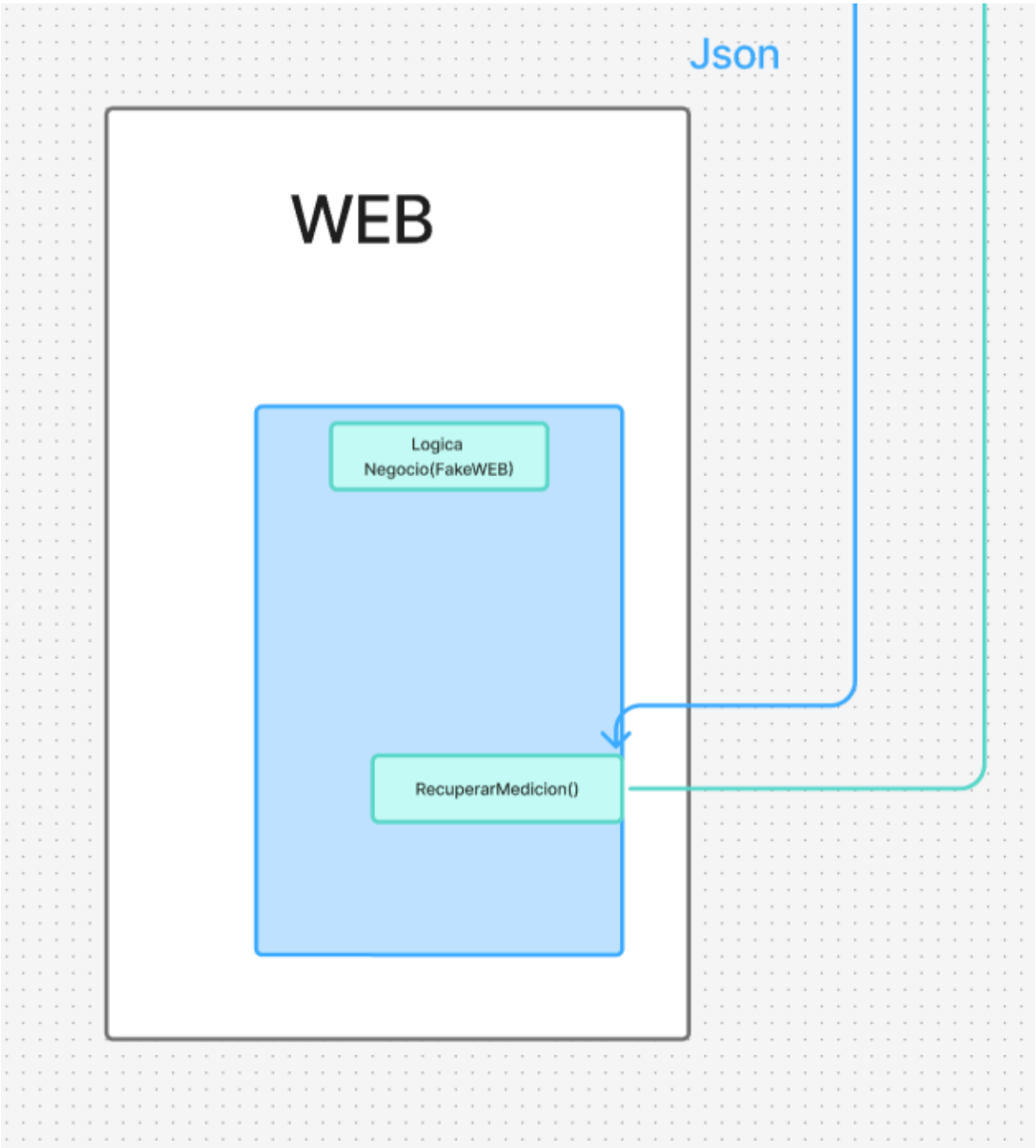
Quiero mantener esta lógica sencilla y no añadir más funciones por ahora.

Genera el código en Java, preparado para usarlo en Android Studio.

- Añade comentarios explicativos en las funciones principales.
- Incluye una pequeña prueba automática (un test unitario o método de prueba) para simular el envío y recepción de datos.

5. Lógica Fake (Cliente Web)

Prompt:



Estoy haciendo un proyecto de Biometría y Medio Ambiente.

Ahora quiero que generes únicamente el código de la lógica fake de la parte WEB, teniendo en cuenta el siguiente dibujo.

Esta lógica se ejecutará en el navegador usando JavaScript.

La lógica fake debe estar dentro de una clase o módulo llamado:

LogicaNegocioFakeWEB

Por ahora tendrá únicamente este método:

recuperarMedicion()

El funcionamiento será:

Servidor REST
↓
JSON
↓
recuperarMedicion()
↓
WEB

La medición recibida tiene esta estructura:

Medicion(id, fecha, tipo, valor, major, minor)

recuperarMedicion() debe obtener del servidor la última medición mediante la ruta GET correspondiente, recibirla en formato JSON y dejar los datos preparados para que puedan mostrarse posteriormente en la interfaz web.

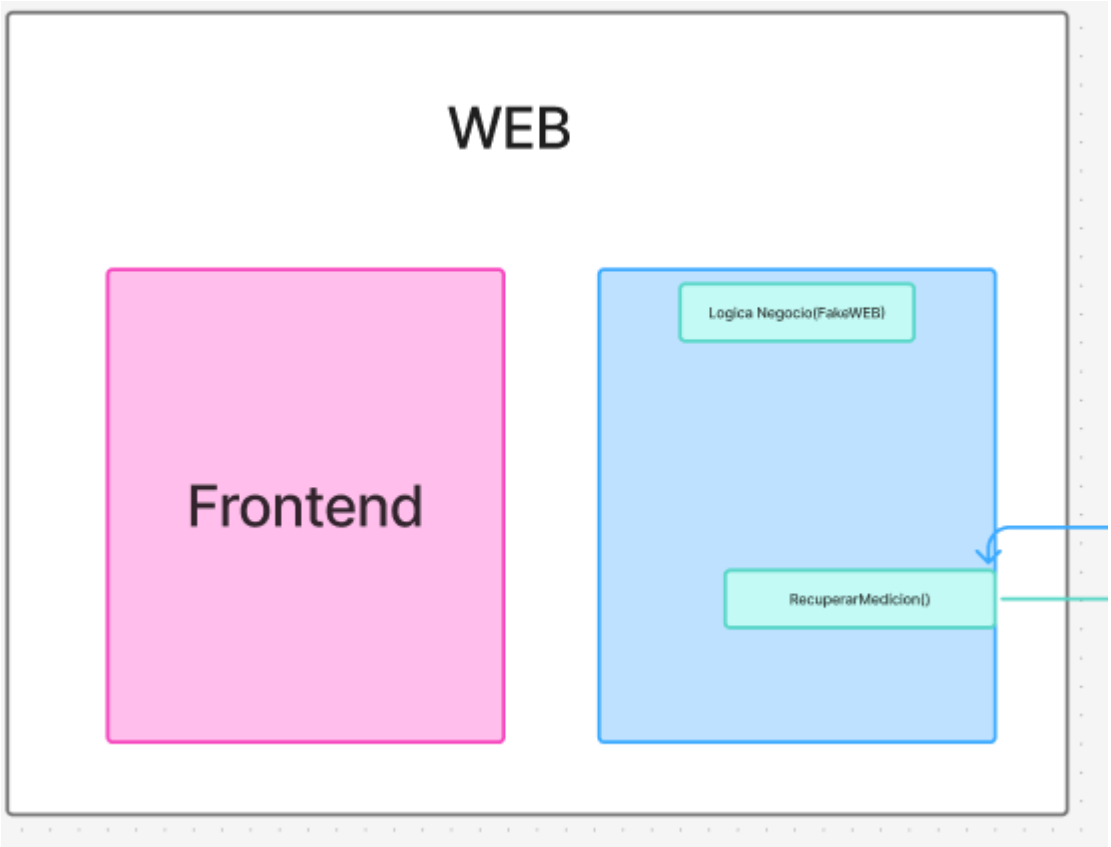
No quiero añadir más funciones por ahora.

Genera únicamente el código JavaScript correspondiente a esta lógica fake para ejecutarlo en el navegador.

- Incluye comentarios explicativos en el código JavaScript.
- Añade una pequeña prueba automática (como una llamada de prueba invocando recuperarMedicion() y verificando la respuesta por consola) para validar la integración.

6. Diseño de Interfaz Web

Prompt:



Ultima medicion:

ID : 50

Tipo : 1

Valor : 70

Obtener ultima medicion

Estoy haciendo un proyecto de Biometría y Medio Ambiente.

Ahora quiero que generes únicamente el diseño de la UX de la parte WEB, teniendo en cuenta el diseño de la imagen que te he pasado.

La web se hará en PHP y quiero que sea muy sencilla.

La pantalla debe tener:

- un header
- un título grande que diga “Última medición:”;
- debajo, mostrar únicamente:
  - ID de la medición;
  - Tipo;
  - Valor;
- un botón debajo de los datos que diga “Obtener última medición”.

La medición tiene esta estructura:

Medicion(id, fecha, tipo, valor, major, minor)

pero en la interfaz solo quiero mostrar:

- id;
- tipo;
- valor.

El botón “Obtener última medición” debe servir para obtener los datos mediante recuperarMedicion() y actualizar la información mostrada. tambien al cargar la pagina se obiene la ultima medicion.

Quiero que el diseño sea igual de simple que el de la imagen, sin gráficas, menús, tarjetas ni elementos adicionales.

Genera únicamente el código PHP, HTML y CSS necesario para reproducir este diseño de forma sencilla.

Añade comentarios breves.